# WRITING AND UNDERSTANDING DOCKERFILES

---

## DOCKERFILE

1. Dockerfile is a text document that contains all the commands a user could called on command line to assemble an image
2. Dockerfile is a file that having all the dependencies to create a image with the help of this image we create a container




## DOCKERFILE INSTRUCTIONS

1. FROM -------> Defines the base image for the Docker image.
2. RUN ---------> Executes commands during image build time.
3. CMD --------> Specifies the default command to run when the container starts.
4. ENTRYPOINT ----> Sets the main command that always runs in the container.
5. LABEL ------> Adds metadata information to the image.
6. MAINTAINER ----> Defines the author of the image
7. EXPOSE -----> Declares the port the container listens on.
8. ENV ----------> Sets environment variables inside the container. (key-value form)
9. ADD ---------> Copies files and can extract compressed files from source to container.
10. COPY ----------> Copies files/directories from host to container.
11. WORKDIR ----> Sets the working directory inside the container.
12. VOLUME -----> Creates a mount point for persistent storage
13. USER ----------> Specifies the user to run commands inside the container.
14. ARG ----------> Defines build-time variables.

## DOCKERFILE-1 (HTTPD)

1. vim Dockerfile

```
FROM ubuntu:latest
RUN apt update && \
      apt install apache2 -y
COPY index.html /var/www/html/index.html
EXPOSE 80
```

CMD [“apache2ctl” , “-D” , “FOREGROUND”]

2. vim index.html
   <h1> These is my httpd container </h1>
3. docker build -t httpd .
4. docker images
5. docker run -d -p 8081:80 –name httpd –rm httpd
6. docker ps

**PUSH ON DOCKERHUB**

1. docker tag httpd mayurpandit25/httpd:v1
2. docker login
3. docker push mayurpandit25/httpd:v1

---

**DOCKERFILE-2 (TOMCAT)**

1. vim Dockerfile

- FROM ubuntu:22.04
- RUN apt update && \
  apt install openjdk-11-jdk -y
- ADD https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.118/bin/apache-tomcat-9.0.118.tar.gz /opt/
- WORKDIR /opt/
- RUN tar -xzf apache-tomcat-9.0.118.tar.gz
- COPY student.war /opt/apache-tomcat-9.0.118/webapps/
- EXPOSE 8080
- CMD ["/opt/apache-tomcat-9.0.118/bin/catalina.sh", "run"]

2. docker build -t tomcat .
3. docker images
4. docker run -d -p 8080:8080 –name tomcat –rm tomcat

---

**DOCKERFILE-3 (NGINX)**

1. vim Dockerfile

- FROM ubuntu:latest
- RUN apt update && \
  apt install nginx -y

- COPY index.html /var/www/html/index.html
- EXPOSE 80
- CMD [“nginx”, “-g”, “daemon off;”]

3. docker build -t nginx .
4. docker images
5. docker run -d -p 80:80 –name nginx –rm nginx

---

**DOCKERFILE-4 (Node.js App)**

1.vim Dockerfile

- FROM node:18
- WORKDIR /app
- COPY package*.json ./
- RUN npm install
- COPY . .
- EXPOSE 3000
- CMD ["node", "server.js"]

2. docker build -t node-app:v1 .
3. docker run -d --name node-app -p 3000:3000 node-app:v1
4. http://localhost:3000

---

**DOCKERFILE-5 (PYTHON)**

1. vim Dockerfile

- FROM python:3.10
- WORKDIR /app
- COPY requirements.txt .
- RUN pip install -r requirements.txt
- COPY . .
- EXPOSE 5000
- CMD ["python", "app.py"]

2. docker build -t flask-app:v1 .
3. docker run -d --name flask-app -p 5000:5000 flask-app:v1
4. http://localhost:5000

---

**DOCKERFILE-6 (MYSQL)**

1. vim Dockerfile

- FROM mysql:8
- ENV MYSQL_ROOT_PASSWORD=root
- ENV MYSQL_DATABASE=testdb
- COPY init.sql /docker-entrypoint-initdb.d/

2. docker build -t mysql:v1 .
3. docker run -d --name mysql-db -p 3306:3306 mysql:v1
4. docker logs mysql-db

---

**TASK:**

1. create a dockerfile for nginx/apache2/tomcat and take base image amazon linux
2. create a dockerfile and get the nginx/apache2 as a base image

---

**MULTI-STAGE DOCKERFILE**

1. Multi-stage Dockerfile is mainly used to create a lightweight Docker image.
2. In this method, we divide the Dockerfile into multiple stages.
    - First stage → used for building the application.
    - Second stage → used for running the application.
3. This helps reduce image size because unnecessary build tools are not included in the final image.

**PRACTICAL**

1. git clone https://github.com/mayurpandit25/studentapp-ui.git
2. vim Dockerfile

# STAGE 1: Build WAR using Maven

- FROM maven:latest AS bobthebuilder
- COPY studentapp-ui /studentapp-ui
- WORKDIR /studentapp-ui
- RUN mvn clean package

# STAGE 2: Deploy WAR on Tomcat

- FROM tomcat:jre25-temurin-noble
- COPY --from=bobthebuilder /studentapp-ui/target/*.war /usr/local/tomcat/webapps/student.war
- EXPOSE 8080
- CMD ["catalina.sh", "run"]

3. docker build -t student .
4. docker images
5. docker run -d -p 8080:8080 --name tomcat student {public_ip:8080/student/

---